

CMSE 801 - Day 9



More modeling with ordinary differential equations, including using `solve_ivp`

September 25, 2024

General Course Announcements

- **Reminder:** Homework 1 is due this Friday, September 27.
 - *Don't wait until the last minute!* -- It shouldn't be overly challenging but it does take time to work through all of the components, which are designed to make sure you review *all* of the fundamentals we've covered up to this point.
- Also, we will have **our first quiz in class a week from Monday, September 30.**
 - This will cover concepts that are covered by Homework 1 (so make sure you're feeling good about your Python fundamentals).
- Homework 2 has been released **today**. It will be due Friday, **October 11.**
 - This will cover using Matplotlib and NumPy as well as a little bit of ODE content (which we're covering this week).

Questions/comments from Day 9 PCA

- What is `solve_ivp` doing? **We'll talk about this.**
- One of the most challenging pieces was just figuring out how to use `solve_ivp` for solving an initial value problem
 - "It seems like `solve_ivp` makes things more complicated than they need to be!"
 - Careful attention must be paid to how you need to write your code in a way that allows `solve_ivp` to use it. **We'll break this down**
- Some challenges with making sure the right pieces of the code were updated correctly, including making sure the derivatives function was correctly defined for the air resistance component -- **you'll get to see the full solutions!**

`solve_ivp` is finding the numerical solution to the system of ODEs just like you did with the "update equations"

Let's take a quick look at [the wikipedia page about numerical solving ODEs](#) and the `solve_ivp` [documentation](#).

Let's take a look at what `solve_ivp` needs in order to solve a systems of ODEs

```
solution = solve_ivp(deriv_func, (t_start, t_stop), init_y, t_eval=time, args=[arg1, ...])
```

↓

Solution:
solution.y is
array containing
value of y for each
defined time,
with initial value in
first row

↓

Name of function that
returns derivatives of y
(instantaneous change of y for
change of t)

↓

Start and stop time
for solution

↓

Array of initial
value(s) of y

↓

Array of all the times
at which you want
the values of y

↓

Optional arguments if
deriv_func()
needs extra variables to
execute

Of course, `solve_ivp` will expect the function that calculates the derivatives to be in a specific format as well

```
In [ ]: def derivs(time, curr_vals):  
  
    # Declare parameters  
    g = -9.81 # m/s^2  
    A = 0.4 # m^2  
    m = 80.0 # kg  
  
    # Unpack the current values of the variables we wish to "update" from  
    # Order matters! (needs to match the order of your initial values)  
    h, v = curr_vals  
  
    # Right-hand side of odes, which are used to compute the derivative  
    dhdt = v  
    dvdt = g + (-0.65 * A * v * abs(v))/m  
  
    return dhdt, dvdt # Again, order matters, needs to match order of in
```

Or, we could have written this such that the arguments can be passed into the derivatives function

You don't **have** to do this, but it can make the process of exploring the effect of parameters on your model a bit easier. You'll see an example of this in the Day 10 PCA.

```
In [ ]: def derivs(time, curr_vals, g, A, m):  
  
    # Unpack the current values of the variables we wish to "update" from  
    # Order matters! (needs to match the order of your initial values)  
    h, v = curr_vals  
  
    # Right-hand side of odes, which are used to compute the derivative  
    dhdt = v  
    dvdt = g + (-0.65 * A * v * abs(v))/m  
  
    return dhdt, dvdt # Again, order matters, needs to match order of in
```

Solutions to the problems you were asked to solve in the PCA are in the ICA for today.

Let's take a look at [those](#) real quick.

Goals for today

- Try to use what you learned in the PCA and what we just dicussed to solve a different, but similar problem.
- Compare your "update equation" solution to the solution provided by `solve_ivp`

Looking forward - Day 10 PCA: Exploring the concept of a "compartmental model" and how ODEs can be used for these types of model

- You'll look at a "Viral Kinetics" model, which is a compartmental model used to explore how viruses can multiply within your body
- You'll explore how this model can be turned into code and how `solve_ivp` can be used to explore how the model evolves.

